

WHIR on Bitcoin Script

A formal account of the STIR and WHIR proximity tests,
and a review of bitcoin-stark-verifier against them

Subject: AppliedPQC/bitcoin-stark-verifier · Sources: ePrint 2024/390, 2024/1586 · 78
tests passing · D1 / D2 / D3 fixed, query constrained and threaded

SUMMARY OF FINDINGS

- **The primitives are sound and genuinely verified.** Poseidon2, the duplex sponge, the challenger, the sumcheck round and multilinear evaluation are each checked against a Rust transcription of Plonky3, with negative cases. A real Plonky3 WHIR proof's Merkle opening authenticates in script, and a tampered leaf is rejected.
- **Three defects sat above them,** all in the composition rather than the arithmetic: Fiat-Shamir challenges were never derived (**D1**), the closing identity was a tautology (**D2**), and the composed `verify()` was a cost model that was never executed (**D3**). All three are fixed, with adversarial tests.
- **The repair was cost-negative.** Deriving challenges in script replaced a spurious second permutation per sumcheck round with a correct single one. Transcript cost fell from **67 to 44** permutations (predicted ≈ 46) — 5.0% of total script weight down to **3.3%** — against 1300 permutations of Merkle work, unchanged.
- **The query is now constrained.** The opened row is bound to its leaf, a round's answers are batched into the next target, and the accumulated constraints are now *derived* rather than supplied — see §15.5. The bookkeeping turned out to be nearly free, and finding that out corrected two things this report had wrong.
- **Two configuration levers remain unused,** worth roughly 20% of script weight between them: `pow_bits = 0` where both reference implementations use ≈ 22 , and the choice of soundness regime, currently the conjectural one.

Part I The algorithm, formally

1 Setting and notation

Fix a finite field $\mathbb{F}$ with $\text{char}(\mathbb{F}) \neq 2$ and a *smooth* evaluation domain $\mathcal{L} \subseteq \mathbb{F}^*$ — a multiplicative coset whose order is a power of two. For a degree bound d ,

$$\text{RS}[\mathbb{F}, \mathcal{L}, d] = \{ f : \mathcal{L} \rightarrow \mathbb{F} \mid \exists \hat{g} \in \mathbb{F}^{<d}[X], f(x) = \hat{g}(x) \forall x \in \mathcal{L} \}, \quad \rho := d/|\mathcal{L}|.$$

When $d = 2^m$ the code has an equivalent multilinear reading, and this is the view both protocols use. Writing $\text{pow}(x, m) = (x^{2^0}, \dots, x^{2^{m-1}})$,

$$\text{RS}[\mathbb{F}, \mathcal{L}, m] = \{ f \mid \exists \hat{f} \in \mathbb{F}^{<2}[X_1, \dots, X_m], f(x) = \hat{f}(\text{pow}(x, m)) \}.$$

$\Delta(f, g)$ is fractional Hamming distance and $\Lambda(\mathcal{C}, f, \delta)$ the list. The equality polynomial is

$$\text{eq}(\mathbf{X}, \mathbf{Y}) = \prod_{i=1}^m (X_i Y_i + (1 - X_i)(1 - Y_i)), \quad \hat{f}(\mathbf{z}) = \sum_{\mathbf{b} \in \{0,1\}^m} \hat{f}(\mathbf{b}) \cdot \text{eq}(\mathbf{b}, \mathbf{z}).$$

2 Constrained Reed–Solomon codes

WHIR’s object of study is not RS but a subcode carrying one sumcheck-shaped constraint:

$$\text{CRS}[\mathbb{F}, \mathcal{L}, m, \hat{w}, \sigma] = \left\{ f \in \text{RS}[\mathbb{F}, \mathcal{L}, m] : \sum_{\mathbf{b} \in \{0,1\}^m} \hat{w}(\hat{f}(\mathbf{b}), \mathbf{b}) = \sigma \right\}. \quad (\text{Def. 1})$$

The weight polynomial

Definition 1 leans on $\hat{w}$ without saying what it is, and it is worth being precise, because every later step is a statement about $\hat{w}$ and almost all of the implementation work is bookkeeping for it.

$\hat{w}$ is a polynomial in $1 + m$ variables,

$$\hat{w}(Z, \mathbf{X}) = \hat{w}(Z, X_1, \dots, X_m),$$

whose first argument Z is a *placeholder for a value of the polynomial being tested* and whose remaining arguments $\mathbf{X}$ range over positions on the hypercube. Substituting $Z \mapsto \hat{f}(\mathbf{b})$ and $\mathbf{X} \mapsto \mathbf{b}$ and summing over $\mathbf{b} \in \{0,1\}^m$ turns it into a single field element, and Definition 1 asks that element to be σ . So $\hat{w}$ is not a constraint on f pointwise — it is the *shape* of one aggregate linear statement about f , and σ is its claimed value.

Two restrictions make it usable rather than merely general.

(i) **Linear in Z .** Every $\hat{w}$ WHIR forms has the shape

$$\hat{w}(Z, \mathbf{X}) = Z \cdot w(\mathbf{X}), \quad w \text{ multilinear in } \mathbf{X},$$

so the sum $\sum_{\mathbf{b}} \hat{f}(\mathbf{b}) w(\mathbf{b})$ is a sumcheck over a *product of two multilinear*s. That is what fixes the degree of every round polynomial at two, which in turn is why a round sends three values — and why, with $\hat{h}(0) + \hat{h}(1) = \sigma$ pinning one of them, only $\hat{h}(0)$ and $\hat{h}(\infty)$ travel. The entire cost model of Section 12 follows from this one line.

(ii) **Closed under the recursion.** Section 6’s update

$$\hat{w}'(Z, \mathbf{X}) = \hat{w}(Z, \boldsymbol{\alpha}, \mathbf{X}) + Z \cdot \sum_{i=0}^t \gamma^{i+1} \text{eq}(\mathbf{z}_i, \mathbf{X})$$

is linear in Z whenever $\hat{w}$ is, since binding $\boldsymbol{\alpha}$ does not touch Z and the added term is visibly Z times something. So $\hat{w}'$ is again of the form $Z \cdot w'(\mathbf{X})$, and the protocol never leaves the class it started in.

Unfolding (ii) across all M rounds gives the concrete object the verifier ends up holding:

$$w(\mathbf{X}) = \sum_j c_j \cdot \text{eq}(\mathbf{z}_j, \mathbf{X}),$$

a **weighted sum of equality polynomials** — that is, nothing more than a list of pairs $(c_j, \mathbf{z}_j)$. Every constraint the protocol ever accumulates, from the initial evaluation query through each round’s out-of-domain answer and t shift answers, is one entry in that list, and the coefficients c_j are powers of the batching challenges. §15 is about carrying that list, and §15.4 is about what it costs to evaluate it.

Evaluation as a special case

The point of the definition is that a polynomial *evaluation query* is a special case. Taking $\hat{w}(Z, \mathbf{X}) = Z \cdot \text{eq}(\mathbf{X}, \mathbf{z})$ gives $\sum_{\mathbf{b}} \hat{w}(\hat{f}(\mathbf{b}), \mathbf{b}) = \hat{f}(\mathbf{z})$, so membership with that weight *is* the claim $\hat{f}(\mathbf{z}) = \sigma$. A proximity test for CRS codes is therefore simultaneously a low-degree test and a multilinear polynomial commitment — which is why WHIR replaces FRI, STIR and BaseFold at once rather than sitting beside them.

3 Folding

For $\alpha \in \mathbb{F}$ and $y = x^2$,

$$\text{Fold}(f, \alpha)(x^2) = \frac{f(x) + f(-x)}{2} + \alpha \cdot \frac{f(x) - f(-x)}{2x},$$

extended to α by iteration. The structural fact is that folding is multilinear partial evaluation seen through the code: if $f \in \text{RS}[\mathbb{F}, \mathcal{L}, m]$ then

$$\text{Fold}(f, \alpha) \in \text{RS}[\mathbb{F}, \mathcal{L}^{(2^k)}, m - k], \quad \hat{g}(X_k, \dots, X_m) = \hat{f}(\alpha, X_k, \dots, X_m). \quad (\text{Claim 4.15})$$

4 Rate is the design variable

This is the whole idea behind both protocols, and it is a single line of arithmetic.

Protocol	Degree	Domain	Rate	$\log_2(1/\rho)$ per round
FRI	$d \rightarrow d/k$	$ \mathcal{L} \rightarrow \mathcal{L} /k$	$\rho' = \rho$	unchanged
STIR	$d \rightarrow d/k$	$ \mathcal{L} \rightarrow \mathcal{L} /2$	$\rho' = (2/k)\rho$	$+\log_2 k - 1$
WHIR	$m \rightarrow m - k$	$ \mathcal{L} \rightarrow \mathcal{L} /2$	$\rho' = 2^{1-k}\rho$	$+(k - 1)$

A notational trap. STIR's k is the degree *divisor* (a power of two, $k \geq 4$); WHIR's k is the *number of variables* folded. The two agree under $k_{\text{STIR}} = 2^{k_{\text{WHIR}}}$. The recurrence that appears in code is WHIR's.

The number of queries needed in round i to reach round-by-round error $2^{-\lambda}$ is $t_i = \lceil \lambda / -\log_2(\rho_i + \eta_i) \rceil$ with $\rho_i = 2^{-i(k-1)}\rho$, so $t_i \approx \lambda / (\log_2(1/\rho) + i(k-1))$. The sum over $M = m/k$ rounds is harmonic, not linear; using $\sum_{i=1}^M 1/(i+c) < \log(M/c+1) + 1$ (STIR Fact C.1),

$$q = O\left(\frac{\lambda}{\log(1/\rho)} + \frac{\lambda}{k} \log\left(\frac{m}{k \log(1/\rho)}\right) + \frac{m}{k}\right) \quad \text{versus FRI's} \quad O\left(\lambda + \frac{\lambda m}{k}\right).$$

In the typical regime $m \leq \lambda$, $\rho = O(1)$, setting $k \approx \log m$ yields $q = O(\lambda)$, which is optimal. **There is no further protocol-level lever on query count.**

5 A STIR iteration: quotient and degree correction

STIR sends a folded g on a larger domain, pins it with an out-of-domain sample, checks it against t shift queries, and forms the next function as a quotient:

$$\text{Quotient}(f, S, \text{Ans}, \text{Fill})(x) = \frac{f(x) - \widehat{\text{Ans}}(x)}{\hat{V}_S(x)} \quad (x \notin S).$$

Quotienting by a *wrong* value pushes the result far from low degree (Lemma 4.4). The price is that the degree drops by $|\mathcal{G}|$, so the round closes with degree correction — a geometric sum evaluated in $O(\log e)$ operations by repeated squaring.

6 A WHIR iteration: constraints instead of quotients

1. **Sumcheck rounds.** k rounds on $\sum_{\mathbf{b}} \hat{w}(\hat{f}(\mathbf{b}), \mathbf{b}) = \sigma$, giving $\hat{h}_{1..k}$ and randomness α .
2. **Folded function.** The prover sends g on $\mathcal{L}^{(2)}$.
3. **Out-of-domain sample.** $z_0 \leftarrow \mathbb{F}$, reply y_0 .
4. **Shift queries.** $z_1, \dots, z_t \leftarrow \mathcal{L}^{(2^k)}$, answers $y_i = \text{Fold}(f, \alpha)(z_i)$; then a batching challenge γ .
5. **Recursive claim.**

$$\begin{aligned}\hat{w}'(Z, \mathbf{X}) &:= \hat{w}(Z, \alpha, \mathbf{X}) + Z \cdot \sum_{i=0}^t \gamma^{i+1} \cdot \text{eq}(z_i, \mathbf{X}), \\ \sigma' &:= \hat{h}_k(\alpha_k) + \sum_{i=0}^t \gamma^{i+1} \cdot y_i.\end{aligned}$$

The out-of-domain answer and the t shift answers become $t + 2$ additional *constraints*, folded into the weight polynomial by powers of γ . No quotient, hence no degree correction, hence — as the paper states explicitly — **the verifier performs no divisions at all**. On a machine without OP_DIV, where an inversion must be hinted and checked, that is the reason this project should target WHIR rather than STIR.

7 What a shift query actually is

The most frequently occurring and most easily skimmed quantity in the protocol is r_i^{shift} (WHIR writes z_i). It deserves its own section, because in this implementation it *is* “a query”, and the identification is not obvious.

Which domain it lives in

The first thing to get straight: a shift query is **not** drawn from the new domain $\mathcal{L}'$ the prover just committed to. It is drawn from

$$r_i^{\text{shift}} \leftarrow \mathcal{L}^{(2^k)} = \{x^{2^k} : x \in \mathcal{L}\}, \quad |\mathcal{L}^{(2^k)}| = |\mathcal{L}|/2^k,$$

the domain the *folded* function lives on. And the folded function is **virtual** — no one sends it. To read it at a point the verifier must read the original f across the whole fibre:

$$y_i = \text{Fold}(f, \alpha)(r_i^{\text{shift}}), \quad \text{requiring } f \text{ on } \{x : x^{2^k} = r_i^{\text{shift}}\}, \text{ which is } 2^k \text{ values.}$$

Those 2^k values are the paper’s “alphabet $\mathbb{F}^{2^k}$ ” — one query reads one symbol, and a symbol is 2^k field elements. In the implementation it is a Merkle **row**, and it is exactly the row that has to be bound to its leaf.

Why it has to exist

This is the pivot of the whole STIR/WHIR construction, and the rate reduction of Section 4 is what it pays for. In FRI the folded function lives *naturally* on $\mathcal{L}^{(k)}$ and the verifier can compute it unaided. To lower the rate, STIR and WHIR have the prover re-commit a g on a *larger* new domain.

But nothing forces g to be $\text{Fold}(f, \alpha)$. The prover could send an unrelated low-degree polynomial. **The shift queries are the pin that fastens the new oracle back to the**

old one: they require $\hat{g}(r_i^{\text{shift}}) = \text{Fold}(f, \alpha)(r_i^{\text{shift}})$ at t random points. Hence the name: STIR = *Shift To Improve Rate*. Out-of-domain sampling does a different job — it pins g to a *unique* codeword in the list; the shift queries then check that this codeword is the fold. Neither suffices alone.

It is the soundness term

t is not arbitrary; it is precisely the exponent in the query term of Section 8. The argument is two lines: if the codeword u near g disagrees with $\text{Fold}(f, \alpha)$ on at least a δ fraction, each uniform r_i^{shift} independently lands on a disagreement with probability $\geq \delta$, so

$$\Pr[\text{all } t \text{ points miss}] \leq (1 - \delta)^t = \text{the leading term of } \varepsilon^{\text{shift}}.$$

So $t_i = \lceil \lambda / -\log_2(1 - \delta_i) \rceil$, and rate reduction saves money precisely because it raises the usable δ and so lowers t . Section 4’s table and this are two sides of one fact.

What it is in this implementation

In the paper	In the script
sampling r_i^{shift}	<code>challenger::sample(slot)</code> , one base element from the rate
its index in $\mathcal{L}^{(2^k)}$	<code>field::low_bits_to_altstack</code> — these bits are the Merkle path
the 2^k values on the fibre	the opened row, bound to its leaf by <code>merkle::hash_row</code>
$y_i = \text{Fold}(\cdot)(r_i^{\text{shift}})$	<code>multilinear::eval_multilinear(k)</code> folds the row
batching into σ'	<code>constraint::combine_answers(t)</code>
the accumulated pairs	<code>constraint::closing_check</code>

The second row is worth pausing on. **The index is the path** — the sampled bits are directly the direction bits of the Merkle walk, so the spender picks neither which leaf is opened nor where it sits; the binding is structural rather than an added check. It also explains where the depth factor in the cost law comes from: $\text{depth} = \log_2 |\mathcal{L}^{(2^k)}|$, so one shift query is depth Poseidon2 permutations, and that is the origin of the 1300 in Section 9.

8 Soundness, and what it rests on

Compilation through BCS requires *round-by-round* soundness. WHIR’s Theorem 5.2 gives:

$$\begin{aligned} \varepsilon_{i,s}^{\text{fold}} &\leq \frac{d \cdot \ell_{i,s-1}}{|\mathbb{F}|} + \text{err}^*(\mathcal{C}, 2, \delta_i), & \varepsilon_i^{\text{out}} &\leq \frac{2^{m_i} \ell_{i,0}^2}{2|\mathbb{F}|}, \\ \varepsilon_i^{\text{shift}} &\leq (1 - \delta_{i-1})^{t_{i-1}} + \frac{\ell_{i,0}(t_{i-1} + 1)}{|\mathbb{F}|}, & \varepsilon^{\text{fin}} &\leq (1 - \delta_{M-1})^{t_{M-1}}. \end{aligned}$$

Only $\varepsilon^{\text{shift}}$ and ε^{fin} carry the $(1 - \delta)^t$ term; everything else is field-size noise.

Because WHIR discards quotienting, distance preservation under folding is not enough: the analysis needs *list* preservation, which is the paper’s new tool.

Definition 4.9 (mutual correlated agreement, informal). Gen has mutual correlated agreement if, with high probability over α : for *every* set S on which $\sum_j r_j f_j$ agrees with $\mathcal{C}$, *every* f_i agrees with $\mathcal{C}$ on S .

Standard correlated agreement promises only *some* set T ; equating S and T upgrades “folding preserves distance” to “every codeword close to $\text{Fold}(f, \alpha)$ is the folding of a codeword close to f ” (Theorem 4.20). It is proved in the unique-decoding regime and *conjectured* beyond it (Conjecture 4.12).

Regime	Proximity δ	Status	Verifier hashes	Argument
WHIR-UD	$(1 - \rho)/2$	verified	10 000	621 KiB
WHIR-JB	$1 - \sqrt{\rho} - \eta$	Conj. 4.12(1)	5 100	299 KiB
WHIR-CB	$1 - \rho - \eta$	Conj. 4.12(2)	2 700	156 KiB

WHIR Table 5, $m = 24$, $\rho = 1/2$, $\lambda = 128$, Goldilocks cubic extension.

Part II Implementation review and repair

9 What the crate is, and its cost law

Two crates: `poseidon2` (the permutation, the quartic extension, Merkle verification over KoalaBear) and `whir` (sponge, challenger, sumcheck, multilinear evaluation, constraint, composed verifier). The design premise is sound and well argued — an algebraic hash makes a digest eight field elements, so compressing two children is sixteen stack items handed to an arithmetic routine, and nothing needs concatenating. That removes the `OP_CAT` soft-fork dependency entirely.

Everything else follows from one measured constant: a Poseidon2 permutation is **572 228 bytes** of script, and nothing else is within 1% of it. So

$$C_{\text{script}} \approx N_{\text{perm}} \cdot c_{\text{perm}}, \quad N_{\text{perm}} = \underbrace{\sum_i t_i \cdot \text{depth}_i}_{\text{Merkle}} + N_{\text{transcript}}.$$

For the crate’s example configuration the braced term is $20 \cdot (21+17+13+9) + 20 \cdot 5 = \mathbf{1300}$ permutations. **Every optimisation that matters is an optimisation of that term.**

10 Verified, versus asserted

A test comparing a script against a Rust transcription establishes that the two *agree*; only a test driven by a real proof, with a negative case, establishes that either is *right*.

Component	Evidence	Negative case	Before	Now
Poseidon2, Merkle compression	vs. reference	—	verified	verified
Duplex sponge, padding	vs. reference	short absorb padded $\neq$	verified	verified
Challenger sampling	vs. reference	coefficient order	verified	verified
Merkle opening of a real proof	real commitment root	tampered leaf rejected	verified	verified
Challenge derivation	squeezed in script	perturbed evaluation rejected	D1	fixed
Closing identity	EF identity	each coefficient rejected	D2	fixed
Composed <code>verify()</code>	executed vs. reference	transcript moves the claim	D3	fixed
Row bound to leaf	<code>hash_row</code> vs. Plonky3	every substitution rejected	gap	fixed
Answers batched into target	vs. reference	every answer moves σ'	gap	fixed
Accumulated pairs threaded	—	—	gap	open

11 The three defects

D1 Fiat–Shamir challenges were never derived from the transcript

The challenges fed to the sumcheck chain and the point at which the final polynomial was evaluated were deterministic constants, invented by the test rather than derived by Plonky3.

Consider one round, with $h(1)$ recovered from $h(0) + h(1) = c$:

$$h(r) = (1 - r) c_0 + r (c - c_0) + r(r - 1) c_\infty.$$

Fix any target c' and any $r \notin \{0, 1\}$. Since $r(r - 1) \neq 0$ is invertible,

$$c_\infty := (c' - (1 - r)c_0 - r(c - c_0)) \cdot (r(r - 1))^{-1}$$

reaches c' for *any* c_0 . Composing, a prover supplying the challenges controls the closing claim exactly. **The soundness error of the chain was 1.**

Fix. `sumcheck_round_fs` absorbs the round's (c_0, c_∞) and squeezes r from the resulting rate, so r is a function of the proof and is fixed only after both evaluations are committed. The invariant is `claim(4) || state(16)` with the sponge on top, already the layout `absorb` wants, so rounds compose.

D2 The closing identity was a tautology, in the base field

The test pushed a , b and ab , rotated twice, and `final_check` recomputed ab to compare against ab — true by construction for any proof. It also compared one coefficient, leaving a base-field equality worth $\leq \log_2 P \approx 31$ bits where the quartic extension exists to provide ≈ 124 .

Fix. `final_check` multiplies in EF and compares all four coefficients; the end-to-end weight is copied off the stack rather than pushed, tying the sumcheck chain to the final polynomial.

D3 `verify()` priced a schedule it could not run

`sample_ef` was never called in `src/`; `absorb` leaves a sixteen-element state where `sumcheck_round` expects `claim c0 c_inf r`; and the test only took `.len()`. Two bugs went with it: `open_query` hardcoded rate slot 0, so every query in a round derived the *same* index and opened the *same* leaf; and a round absorbed two elements where it sends two *extension* elements, then paid a second permutation for a squeeze that absorbing had already performed.

Fix. Wired, executed against a reference, with the slot parameterised and the accounting corrected.

12 The repair's stack discipline

```
// in : claim(4) || state(16)      hints pushed above: c0(4) || c_inf(4)
// out: claim'(4) || state'(16)
{ ext4::copy(0) } { ext4::to_altstack() } // park c_inf -- absorb consumes it
{ ext4::copy(1) } { ext4::to_altstack() } // park c0
{ sponge::absorb(8) } // ONE permutation; absorb IS the duplexing
{ challenger::sample_ef(0) } // r := rate[0..4], bound to the transcript
for _ in 0..4 { { 23 } OP_ROLL } // lift claim over state' (2*D + WIDTH - 1)
```



```

{ ext4::from_altstack() }           // c0
{ ext4::from_altstack() }           // c_inf
for _ in 0..4 { { 15 } OP_ROLL }     // lift r to the top      (4*D - 1)
{ sumcheck::sumcheck_round() }       // consumes claim c0 c_inf r
for _ in 0..16 { { 19 } OP_ROLL }    // restore the invariant  (D + WIDTH - 1)

```

Each OP_ROLL group lifts the deepest slot of its element first, preserving coefficient order.

A depth was wrong on the first attempt: CLAIM_UNDER_STATE_AND_R was written $4D + W - 1 = 31$ where the correct value is $2D + W - 1 = 23$. It surfaced as `InvalidStackOperation` — a class of error that appears only when the script is *executed*, which is itself a footnote to D3.

13 What the repair cost

Rolls and picks are one to two bytes against 572 228 for a permutation, so the wiring is free at this resolution, and the round drops from two permutations to one.

Quantity	Before	Predicted	Measured	Why
Transcript permutations	67	≈ 46	44	absorb is the duplexing
Merkle paths	1300	1300	1300	the term that matters
Transcript share	5.0%	$\approx 3.5\%$	3.3%	—
Tests	—	—	67	all passing

The honest verifier is smaller than the priced skeleton. **There is no efficiency argument for leaving Fiat–Shamir out.** The composed end-to-end script measures **8 967 342 bytes**: a real Plonky3 WHIR proof accepted by one Bitcoin Script execution, with every perturbation of the prover’s evaluations rejected.

14 Constraining the query

Three changes, in the order they had to happen.

The row is bound to its leaf

A query opening has two halves and only one used to be checked. Walking the path proves that *some* committed leaf sits at the sampled index; with the leaf arriving as a hint alongside the row, nothing stopped a spender authenticating the real leaf and folding a different row. `merkle::hash_row` reproduces Plonky3’s `PaddingFreeSponge`: overwrite the first RATE state elements with each chunk and permute, a partial final chunk leaving the rest alone. No padding, no length tag — deliberately *not* the duplex, where zeroing the rest of the rate and binding the count is what stops a short absorb colliding with a padded longer one. Cost: $\lceil \text{row_len}/8 \rceil$ permutations per query.

A round’s answers are batched

`constraint::combine_answers` computes $\sigma' = \hat{h}_k(\alpha_k) + \sum_i \gamma^{i+1} y_i$ by Horner from the top — $t + 1$ multiplications rather than the $2t$ of forming the powers, the outer γ being what shifts every exponent up by one. No permutation at all.

The closing identity, and two readings of it

Each accumulated term collapses, because summing an evaluation against eq *is* evaluation:

$$\sum_b \hat{f}_M(\mathbf{b}) \gamma^e \text{eq}(\mathbf{z}, \mathbf{b}) = \gamma^e \hat{f}_M(\mathbf{z}).$$

So $\hat{w}'$ is exactly the list of $(c_j, \mathbf{z}_j)$ pairs of §2, and checking it means evaluating the final polynomial at each point and taking the weighted sum. That is self-consistent, it is the paper’s reading, and `closing_check` implements it.

It is not the identity Plonky3 checks. There the two polynomials are kept *apart*, each evaluated at its own point:

$$\underbrace{\text{claimed_eval}}_{\text{the chained claim}} = \underbrace{w(\mathbf{R})}_{\text{eval_constraints_poly}} \cdot \underbrace{\hat{f}_M(\mathbf{r}_{\text{fin}})}_{\text{final_evaluations.eval_ext}},$$

and `eval_constraints_poly` evaluates every accumulated `eq( $\mathbf{z}_j, \cdot$ )` explicitly. Since this implementation is checked against Plonky3, that is the shape the script must produce — so an `eq` primitive **is** needed after all. An earlier draft of this report concluded from the collapse above that none was; the conclusion was right about the paper and wrong about the verifier being targeted, and the gap appears only when the two are put side by side.

Measured, since this is where `final_poly_vars` bites:

n_{vars}	bytes per point
2	95,392
3	190,922
4	381,988
5	764,270

Doubling per variable. For the example configuration the accumulated pairs number roughly $\sum_i (t_i + 2) \approx 110$, so at `final_poly_vars` = 4 the closing check is on the order of 42 MB against 743 MB of Merkle work — about 5%. It is also a concrete argument for pushing folding work into earlier rounds: the cost is $n_{\text{points}} \cdot 2^{n_{\text{vars}}}$.

Threading, and why it costs almost nothing

The list of pairs has to reach the closing check having been *derived*, not supplied. The natural guess is that this is expensive: a pair created early should have to be rewritten every round, its coefficient absorbing a factor as the sumcheck binds variables. It does not, and the reason is one line — `eq` factorises across a split,

$$\text{eq}((\mathbf{z}_{\text{pre}}, \mathbf{z}_{\text{suf}}), (\boldsymbol{\alpha}, \mathbf{y})) = \text{eq}(\mathbf{z}_{\text{pre}}, \boldsymbol{\alpha}) \cdot \text{eq}(\mathbf{z}_{\text{suf}}, \mathbf{y}),$$

so binding a prefix never mixes coordinates. Constraints can simply be *accumulated*, and at the end each is evaluated against the concatenation of every round’s folding randomness,

$$\mathbf{R} = \boldsymbol{\alpha}^{(0)} \parallel \boldsymbol{\alpha}^{(1)} \parallel \dots \parallel \boldsymbol{\alpha}^{(\text{fin})},$$

a constraint of arity n_c reading the **last** n_c **coordinates** of $\mathbf{R}$ — which is what it means for the randomness from round i onwards to be what binds a constraint born at round i .

On a stack that rule is free. Push $\mathbf{R}$ with R_0 deepest and the last n_c coordinates are the *topmost* n_c : every constraint reads from the top, and a longer one reaches further down. No slicing, no offset arithmetic, no per-round rewriting. Keeping each round’s challenge rather than discarding it costs **76 bytes**, 0.011% of a round.

A second convenience sits in the point expansion. An out-of-domain sample is a scalar u whose constraint point is $(u^{2^{m-1}}, \dots, u^2, u)$; repeated squaring produces u first, so moving the block to the altstack top-first lands the coordinates in exactly the order the `eq` evaluation consumes them. The two conventions meet without a roll.

Primitive	What it is	Cost
eq_eval	$\prod_i (1 + 2z_i r_i - z_i - r_i)$	$2n$ ext. mult.
expand_univariate	the square-power point	$n - 1$ ext. mult.
constraint_eval	$w(\mathbf{R})$	per constraint, above
sumcheck_rounds_fs_keep	buries each round's α	76 B per round

None costs a permutation. `eq_eval` and `expand_univariate` are checked against Plonky3's own `Point::eval_eq` and `Point::expand_from_univariate`, not only against a local reference — either could otherwise have been a consistent misreading.

An absorb four times too narrow

Wiring the threading in exposed a defect the earlier accounting had hidden. Out-of-domain answers and the final polynomial are *extension* elements, and Plonky3 observes them with `observe_algebra_slice` — four base elements each, not one. The out-of-domain points are also sampled one at a time *between* the answers rather than once after them, so a round costs one duplexing per sample and not one per rate block. Transcript permutations for the example configuration move from 44 to **54** and the share from 3.3% to **4.0%**. It is a correction in the expensive direction, and the second time this accounting has moved after the schedule was *executed* rather than priced.

15 Does it fit in a Bitcoin transaction?

Every figure so far is a script length. This is what those lengths mean, priced against a `WhirConfig` derived by Plonky3's own security analysis rather than a schedule written out by hand, so the query counts and domain sizes are whatever the analysis actually produces.

Neither of the two limits that bind is the one usually named: Taproot removed the 10 000-byte script cap and the 201-opcode cap, so what is left is *weight* — 400 000 units for a standard transaction, 4 000 000 for a block, witness bytes counting one unit each.

A single Poseidon2 permutation is 572 228 bytes, so a standard transaction holds **0.70** of one and an entire block holds **6.99**. **The unit that does not fit is not the query — it is one hash.**

λ	pow	m	rounds	queries	depth	one query
80	0	16	2	21	16	13.7 MB = 3.43 blocks
80	22	16	2	15	16	13.7 MB = 3.43 blocks
100	22	16	2	20	16	13.7 MB = 3.43 blocks
80	22	20	3	15	20	16.0 MB = 4.01 blocks
80	22	24	4	15	24	18.3 MB = 4.58 blocks

The first three rows are the answer to a question worth separating out. **Lowering the security level buys fewer queries, not cheaper ones.** At $m = 16$ the cost of one query is identical at 80 and 100 bits, because depth is set by the domain size and not by λ ; what moves is the count, 20 down to 15. So no security level puts a query in a transaction — a query is three to five *blocks* throughout.

For the whole verifier the level does buy something, roughly proportionally:

λ	pow	m	permutations	bytes	blocks
100	22	16	653	373,664,884	93.4
80	22	16	497	284,397,316	71.1
80	22	20	708	405,137,424	101.3
80	22	24	934	534,460,952	133.6

76% of the cost for 80% of the security, which is close to linear because query count is close to linear in λ and query count is very nearly the whole bill.

The same table settles §16’s proof-of-work point more sharply than the small configuration could. At `pow_bits = 0` neither level is derivable at all beyond a toy instance: 100-bit at $m = 16$ already requires 19 bits of grinding, $m = 20$ requires 28, and $m = 24$ requires 37. Even 80-bit needs 8 bits at $m = 20$. `pow_bits: 0` is therefore not a conservative choice — it is a configuration that mostly does not exist.

What this means for the next step. Chunking is not optional and its unit has to be *sub-query*. The natural seam is one Merkle level — 572 252 bytes, 0.14 of a block — with the intermediate digest committed between chunks, which is the shape BitVM2’s disprove pattern already has. Nothing in §15 changes this: threading is arithmetic, and arithmetic is free at this resolution.

16 Two unused levers

Proof of work

`pow_bits: 0` is why the security level tops out at 100. Both reference implementations grind: STIR’s benchmarks take 22 bits, reducing the soundness required from the IOP from 2^{-128} to 2^{-106} ; WHIR uses $m - \log \rho - 3$, also 22 at $m = 24$, $\rho = 1/2$. The trade is unusually favourable here — a query costs $\text{depth} \times 572 \text{ KB}$, a proof-of-work check costs one permutation regardless of depth:

$$\Delta N_{\text{perm}} = -\lceil 20/4 \rceil \cdot 9 + 1 = -44 \text{ of } 234 \approx \mathbf{19\% \text{ of the dominant term.}}$$

Soundness regime

`SecurityAssumption::CapacityBound` is the cheapest of the three by a factor of 3.7 in verifier hashes — here, directly, in script bytes — and the only one resting on the unproven half of Conjecture 4.12. A defensible choice, and the papers’ own benchmarking default, but a verifier that settles bitcoin should say out loud which assumption it stands on.

Part III Completeness and soundness

This part separates two things: the completeness and soundness of the **protocol** (the papers’ results, stated here and explained in shape), and of **this script** (this repository’s responsibility, covering only what it actually checks). Conflating them is the error this report has repeatedly pointed at.

17 Completeness

Proposition (protocol). If $f \in \text{CRS}[\mathbb{F}, \mathcal{L}, m, \hat{w}, \sigma]$ and the prover is honest, the verifier accepts with probability 1.

Proof sketch. Each equality is an identity in the honest case, not a probabilistic event.

1. **Sumcheck.** The honest $\hat{h}_\ell(X) = \sum_{\mathbf{b}} \hat{w}(\hat{f}(\alpha, X, \mathbf{b}), \dots)$ makes $\hat{h}_1(0) + \hat{h}_1(1) = \sigma$ and $\hat{h}_\ell(0) + \hat{h}_\ell(1) = \hat{h}_{\ell-1}(\alpha_{\ell-1})$ hold by the definition of the sum.
2. **Folding preserves codewords.** By Claim 4.15, $f \in \text{RS}$ implies $\text{Fold}(f, \alpha) \in \text{RS}$ with multilinear extension $\hat{f}(\alpha, \cdot)$, so the honest g is a codeword of the right degree on the right domain.
3. **Answers.** The honest prover sets $y_0 = \hat{g}(\mathbf{z}_0)$, and y_i is the Fold value the verifier computes from f itself; the two agree by construction.
4. **Recursive claim.** Substituting (2) and (3) into the definitions of $\hat{w}'$ and σ' gives $\sum_{\mathbf{b}} \hat{w}'(\hat{g}(\mathbf{b}), \mathbf{b}) = \sigma'$, so the inductive hypothesis is preserved.
5. **Closing.** $\hat{f}_M$ is sent in the clear and both final checks hold by construction. $\square$

Proposition (script). Given a proof Plonky3 accepts and a witness laid out as documented, the script runs to completion without aborting.

Why this is measured, not argued. `whir/tests/end_to_end.rs` produces a real proof with Plonky3’s prover, confirms it with Plonky3’s *own* verifier, and only then hands the same object to the script. The order is deliberate: comparing a script against a Rust reference alone would let both pass while misreading a proof in the same way.

18 Soundness

Protocol: round by round

BCS/Fiat–Shamir needs *round-by-round* soundness, not plain soundness: a state function $\text{State}(\text{tr}) \in \{0, 1\}$ that is 1 on the empty transcript iff the statement is true, that a prover cannot flip back once 0, that a verifier’s next challenge flips with probability $\leq \varepsilon_i$, and whose being 0 on a full transcript implies rejection. The strengthening is necessary because Fiat–Shamir lets a prover retry challenges, under which plain soundness does not survive.

The four errors of Section 8 split in two, and the split is useful to an implementer:

Field-size terms ($\varepsilon^{\text{fold}}$, ε^{out} , and the second term of $\varepsilon^{\text{shift}}$): of the form $\text{poly}(2^m, 1/\rho)/|\mathbb{F}|$, shrunk by *extending the field*.

Query terms (the leading $\varepsilon^{\text{shift}}$ and ε^{fin}): $(1 - \delta)^t$, shrunk only by *querying more*, and on Bitcoin each query is depth permutations.

The whole cost structure sits on that line: field size is free (the quartic extension of KoalaBear gives about 124 bits), query count is the only thing that is paid for, and rate reduction exists to lower it.

Script: what it actually guarantees

Property checked	Ground	Error
Challenges unchooseable	r squeezed <i>after</i> (c_0, c_∞) absorbed	see next
One sumcheck round	false claim $\Rightarrow h$ not identical; Schwartz–Zippel	$\leq 3/2^{124}$
Opening is committed	path recomputes the root, <code>OP_EQUALVERIFY</code>	collision
Folded row is the committed row	<code>hash_row</code> reproduces <code>PaddingFreeSponge</code>	preimage
Index unchooseable	index bits <i>are</i> the path direction bits	—
Answers cannot cancel	random linear combination in powers of γ	$\leq (t+1)/2^{124}$
Closing identity	all four EF coefficients; weight taken from the stack	$\leq 1/2^{124}$

Not covered, and so not in that table: threading the accumulated (weight, point) pairs between rounds. Both ends exist — σ' is computed and the closing identity is checked — but the pairs are not yet carried from round to round, so what the script guarantees is that each step holds individually, not WHIR’s full round-by-round soundness. The gap between Section 8’s ε and this table is exactly that.

An assumption that must be named. The query-term errors above are under the capacity bound (`SecurityAssumption::CapacityBound`), which rests on the *unproven* half of Conjecture 4.12. Under the proved unique-decoding setting the query count is $3.7\times$ larger by Section 8’s table, and script bytes scale with it.

Why D1 was “soundness error = 1”

Worth isolating, because it is the line between the first row of that table holding and not. Before the fix r came from the witness, so the prover could see r before choosing (c_0, c_∞) — or choose all three together. For any target c' and any $r \notin \{0, 1\}$,

$$h(r) = (1 - r)c_0 + r(c - c_0) + r(r - 1)c_\infty$$

is an **affine** map in c_∞ whose coefficient $r(r - 1) \neq 0$ is invertible, hence **surjective** onto EF: pick any c_0 and solve for the unique c_∞ that hits c' . Schwartz–Zippel does not apply — it bounds “fix a polynomial, then sample a random point”, and here the point was not random. Composing across rounds, the closing claim is entirely the prover’s, and the chain’s soundness error is 1 rather than something small. That is why the fix had to bind r to the transcript rather than choose a better constant.

Sources: G. Arnon, A. Chiesa, G. Fenzi, E. Yogev, *STIR: Reed–Solomon Proximity Testing with Fewer Queries*, ePrint 2024/390 (CRYPTO ’24); and WHIR: *Reed–Solomon Proximity Testing with Super-Fast Verification*, ePrint 2024/1586. Implementation figures are from `poseidon2/tests/verifier_cost.rs`, `whir/tests/end_to_end.rs`, `whir/tests/verifier.rs` and `whir/tests/constraint.rs`. The 44 transcript permutations, 3.3% share, 8 967 342-byte composed script, per-point closing-check sizes and 67 passing tests are all measured on this machine; the 135 MB and `eval_multilinear` extrapolations follow the repository README.